

DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

- CLI To-Do List Manager - Technical Specification & Architecture Document 3**
 - 1. Executive Summary & Architecture Overview 3
 - 1.1 Executive Brief 3
 - 1.2 Maturity Assessment 3
 - 1.3 Technical Stack 3
 - 1.4 Architectural Constraints 3
 - 1.5 Critical Dependencies 3
 - 2. Architecture Workflows 4
 - 2.1 Project Implementation Roadmap & Traceability 5
 - 2.2 CLI Command Execution Workflow 5
 - 2.3 Task Management Sequence 6
 - 3. Detailed Technical Specifications & Business Rules 7
 - 3.1 Requirements Traceability 7
 - 3.2 Security Rules 9
 - 3.3 Data Models 10
 - 4. Project Governance & Structural Gaps 10
 - 4.1 Structural Gaps 10
 - 4.2 Remediation & Workflow 10
 - 5. Technical & Domain Glossary (Terminology Reference) 10

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line application designed for efficient task lifecycle management. The system utilizes a service-oriented architecture that strictly decouples the CLI presentation layer, the business logic (service layer), and the data persistence layer (JSON storage). This ensures that task creation, completion, listing, and removal are handled through a predictable and maintainable pipeline.

1.2 Maturity Assessment

The project is structurally sound with a high health index and a clear implementation roadmap. Core infrastructure and early user stories (US1, US2) are complete. The primary remaining gaps are the finalization of User Story 3 and the execution of the polish phase. While formal acceptance criteria and explicit security/performance constraints are absent, the project is deemed READY for final execution given the scope of a local CLI tool.

1.3 Technical Stack

- Python
- pyproject.toml (Project metadata & entry points)
- JSON (Persistence format)

1.4 Architectural Constraints

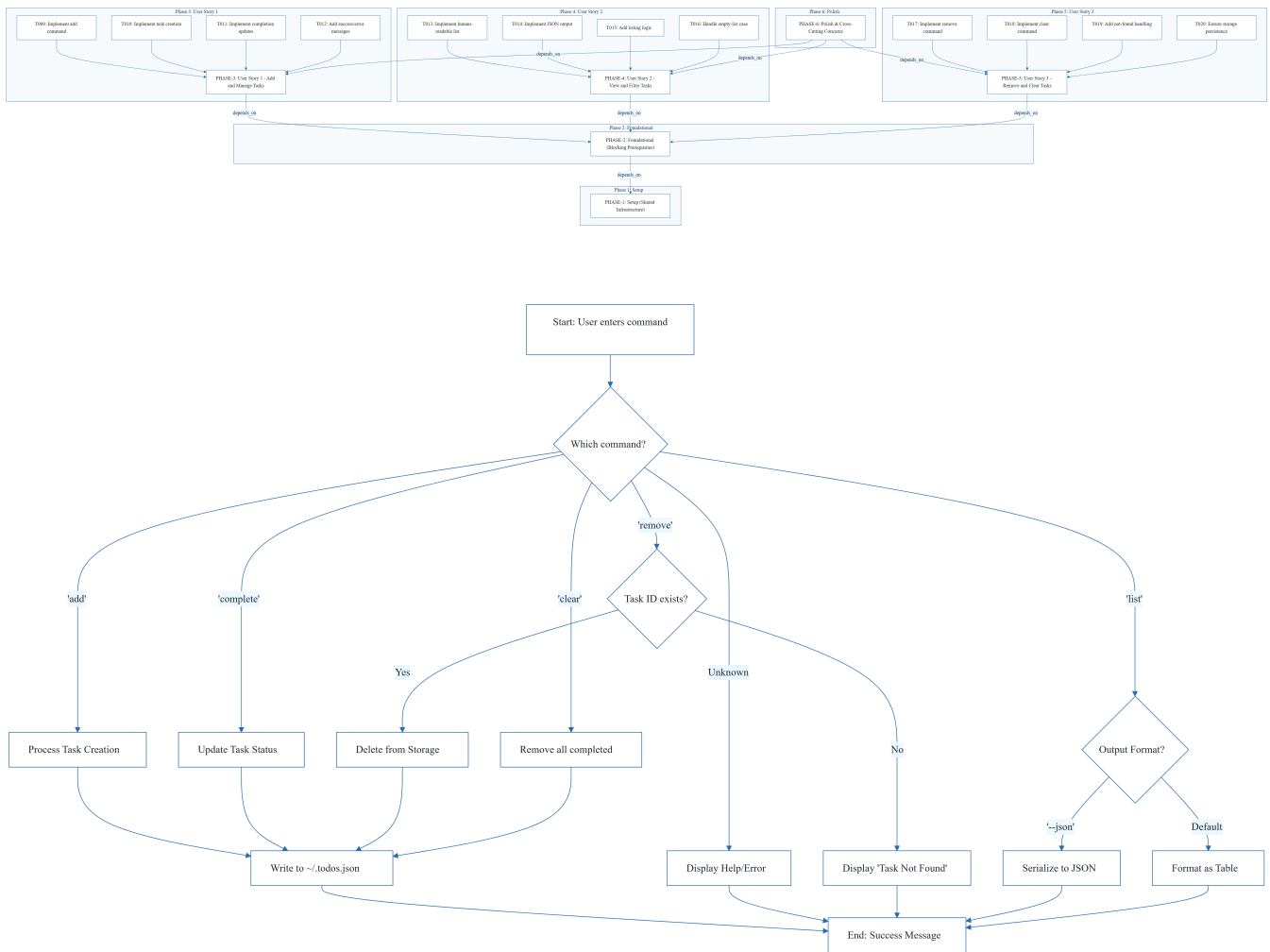
- **Persistence:** All data must be persisted in `~/ . todos . json`.
- **Task Identity:** Mandatory sequential ID assignment for every new task.
- **Auditability:** Mandatory population of `created_at` timestamps upon task creation.
- **Interoperability:** JSON output via the `--json` flag must remain strictly parseable.
- **Execution Pipeline:** Strict linear dependency: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- **Blocking Gate:** The Foundational phase is a mandatory prerequisite for all feature implementation.

1.5 Critical Dependencies

- **File I/O:** Reliable read/write access to `~/ . todos . json`.
- **CLI Configuration:** `pyproject.toml` for correct command-line entry point mapping.

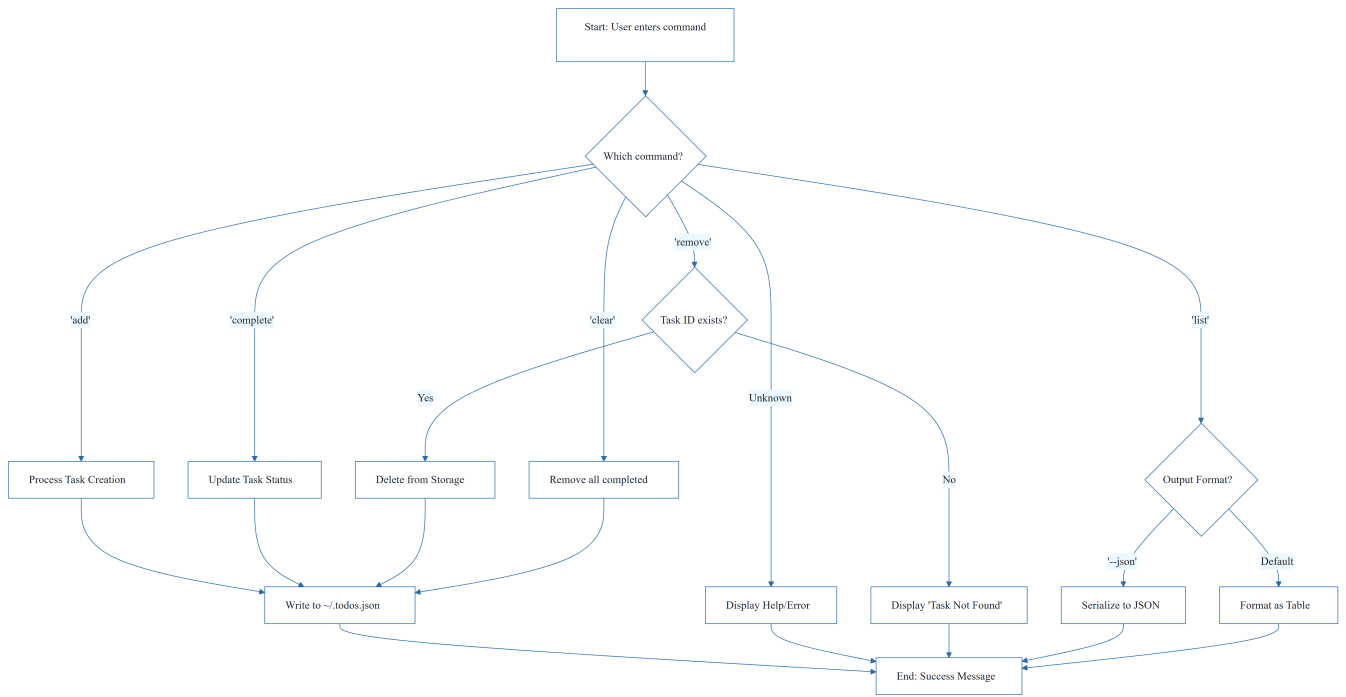
- **Sequential Logic:** Phase 2 (Foundational) must be 100% complete before any User Story work.
- **Data Integrity:** Task serialization helpers in `models.py` are critical for both storage and JSON output.
- **Referential Integrity:** Removal and clear operations must ensure safe persistence without corrupting the JSON structure.

2. Architecture Workflows

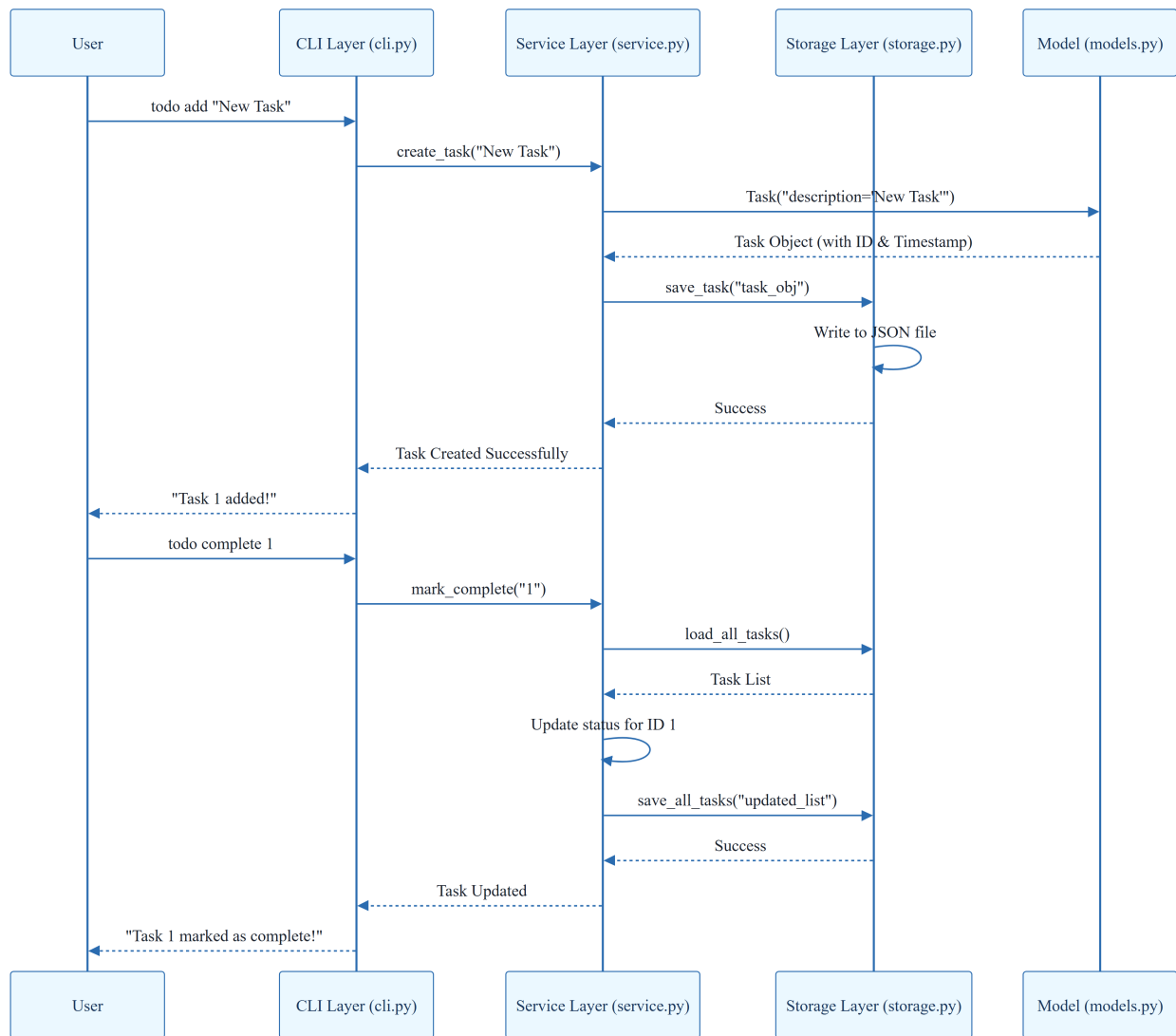




2.2 CLI Command Execution Workflow



2.3 Task Management Sequence



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

ID	Requirement / Task Description	Source Phase	Status
T001	Create Python package skeleton (<code>__init__.py</code> , <code>__main__.py</code> , <code>cli.py</code> , <code>models.py</code> , <code>storage.py</code> , <code>service.py</code>)	PHASE-1	Completed
T002	Create test directory structure (<code>tests/unit/</code> , <code>tests/integration/</code>)	PHASE-1	Completed

ID	Requirement / Task Description	Source Phase	Status
T003	Add project metadata and tooling entry points in <code>pyproject.toml</code>	PHASE-1	Completed
T004	Implement task entity definitions and serialization helpers in <code>models.py</code>	PHASE-2	Completed
T005	Implement JSON storage path resolution and file I/O helpers in <code>storage.py</code>	PHASE-2	Completed
T006	Implement shared command-line parsing and top-level dispatch in <code>cli.py</code>	PHASE-2	Completed
T007	Implement shared service-layer error handling and task collection utilities in <code>service.py</code>	PHASE-2	Completed
T008	Define module entry behavior for <code>python -m todo_manager</code> in <code>__main__.py</code>	PHASE-2	Completed
T009	Implement the add command flow in <code>cli.py</code> and <code>service.py</code>	PHASE-3	Completed
T010	Implement task creation, sequential ID assignment, and <code>created_at</code> population in <code>models.py</code>	PHASE-3	Completed
T011	Implement completion updates for existing tasks in <code>service.py</code>	PHASE-3	Completed
T012	Add user-facing success and error messages for add/complete in <code>cli.py</code>	PHASE-3	Completed
T013	Implement human-readable task listing output in <code>cli.py</code>	PHASE-4	Completed
T014	Implement <code>--json</code> output formatting in <code>cli.py</code> using <code>models.py</code> helpers	PHASE-4	Completed
T015	Add listing logic preserving task order and status fields in <code>service.py</code>	PHASE-4	Completed

ID	Requirement / Task Description	Source Phase	Status
T016	Handle the empty-list case with a clear message in <code>cli.py</code>	PHASE-4	Completed
T017	Implement the <code>remove</code> command flow in <code>cli.py</code> and <code>service.py</code>	PHASE-5	Pending
T018	Implement the <code>clear</code> command flow for removing completed tasks in <code>service.py</code>	PHASE-5	Pending
T019	Add not-found handling for invalid task IDs in <code>cli.py</code>	PHASE-5	Pending
T020	Ensure storage writes persist removals and clears safely in <code>storage.py</code>	PHASE-5	Completed
T021	Document CLI usage and storage behavior in <code>README.md</code>	PHASE-6	Pending
T022	Add quickstart verification notes and examples in <code>quickstart.md</code>	PHASE-6	Pending
T023	Run manual smoke test of all commands against <code>~/todos.json</code>	PHASE-6	Pending
T024	Verify linting and formatting expectations for <code>src/todo_manager/</code>	PHASE-6	Pending
T025	Confirm generated JSON output remains parseable for <code>todo list --json</code>	PHASE-6	Pending
TEST-US1	Validation: <code>todo add</code> and <code>todo complete</code> reflect in stored list	PHASE-3	N/A
TEST-US2	Validation: <code>todo list</code> and <code>todo list --json</code> output correctness	PHASE-4	N/A
TEST-US3	Validation: <code>todo remove</code> and <code>todo clear</code> target correct entities	PHASE-5	N/A

3.2 Security Rules

- **Input Sanitization:** While not explicitly defined, the system must handle invalid task IDs (T019) to prevent application crashes.

- **File Permissions:** The application relies on the user's home directory permissions for `~/todos.json`.

3.3 Data Models

- **Task Entity:**
 - `id`: Integer (Sequential, Unique)
 - `description`: String
 - `completed`: Boolean
 - `created_at`: Timestamp (ISO 8601)

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While 'Independent Tests' exist, formal acceptance criteria per task would improve validation.
Security & Performance Constraints	LOW	Add constraints on maximum JSON file size or input string length sanitization.
Open Questions	LOW	No open questions were identified in the source documentation.

4.2 Remediation & Workflow

The project follows an incremental delivery model. The current workflow requires the completion of Phase 5 (User Story 3) followed by the comprehensive Polish phase (Phase 6) to ensure all cross-cutting concerns (documentation, linting, smoke tests) are addressed before final release.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2	A synchronization gate ensuring all blocking infrastructure is verified before parallel feature development commences.

Term	Category	Context Anchor	Project Definition
Foundational	TECHNICAL_STACK	PHASE-2	The core shared infrastructure layer containing serialization, storage resolution, and dispatch logic required by all subsequent features.
Goal	BUSINESS_DOMAIN	PHASE-3	The high-level functional objective that a specific user story must satisfy to be considered complete.
ID	BUSINESS_DOMAIN	T010	A sequential numeric identifier assigned to each task entity to enable unique referencing and targeted mutation.
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in ~/.todos.json and for scriptable output.
MVP	BUSINESS_DOMAIN	PHASE-3	The minimum viable product consisting of the ability to create and mark tasks as finished.
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The structural grouping of implementation units by user story to ensure independent testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of mandatory design documents including plan, spec, research, data-model, and command contracts.
README	TECHNICAL_STACK	T021	The primary documentation file detailing command-line usage and persistence behavior.
Setup	TECHNICAL_STACK	PHASE-1	The initial phase involving package skeleton creation, directory structuring, and pyproject.toml configuration.
T016	TECHNICAL_STACK	T016	The specific implementation unit responsible for providing a clear user message when the task collection is empty.

Term	Category	Context Anchor	Project Definition
Tests	TECHNICAL_STACK	T002	The verification suite organized into unit and integration directories, validated via quickstart smoke tests.
population in	BUSINESS_DOMAIN	T010	The process of assigning the created_at timestamp during the instantiation of a new task entity.
todo clear	BUSINESS_DOMAIN	T018	The operational command that purges all entities marked as finished from the persistent store.
todo list	BUSINESS_DOMAIN	T013	The operational command that retrieves and displays all stored entities in either human-readable or machine-parseable formats.
■■ CRITICAL	TECHNICAL_STACK	PHASE-2	A blocking constraint indicating that no feature work can proceed until the foundational infrastructure is fully implemented.